

Labolatoria nr 3

Bartłomiej Nałódka

```
(kali㉿kali) - [~/Documents]
$ cat znaki.txt
witam
ąęć trudne znaki!@#
```

```
(kali㉿kali) - [~/Documents]
$ base64 znaki.txt >> nowe.txt
```

```
(kali㉿kali) - [~/Documents]
$ cat nowe.txt
d2l0YW0KxIXEmcW8xIcgdHJlZG5lIHpuYWtpIUAjCg==
```

```
(kali㉿kali) - [~/Documents]
$ base64 -d nowe.txt
witam
ąęć trudne znaki!@#
```

```
(kali㉿kali) - [~/Documents]
$ strings nowe.txt
d2l0YW0KxIXEmcW8xIcgdHJlZG5lIHpuYWtpIUAjCg==
```

Programy `file` i `strings` dają inny efekt - `strings` wyświetla wszystkie możliwe do wyświetlenia znaki, natomiast `file` pokazuje nam typ ich zapisu (jak niżej, opcja `-i` daje więcej detali).

```
(kali㉿kali) - [~/Documents]
$ file -i nowe.txt
nowe.txt: text/plain; charset=us-ascii
```

```
(kali㉿kali) - [~/Documents]
$ file nowe.txt
nowe.txt: ASCII text
```

Zadanie nr 1

Po pobraniu i rozpakowaniu archiwum sprawdzamy jego zawartość:

```
(kali㉿kali) - [~/Downloads]  
$ file File  
File: directory
```

```
(kali㉿kali) - [~/Downloads/File]  
$ file D19910350Lj.pdf  
D19910350Lj.pdf: PDF document, version 1.5, 351 page(s)  
  
(kali㉿kali) - [~/Downloads/File]  
$ file D2020000211201.pdf  
D2020000211201.pdf: PDF document, version 1.5, 18 page(s)  
  
(kali㉿kali) - [~/Downloads/File]  
$ file Text  
Text: ASCII text, with no line terminators
```

Zadanie nr 2

Używając pdftinfo badamy pliki .pdf

```
└─$ pdftinfo D2020000211201.pdf
Title: Ustawa z dnia 28 października 2020 r. o zmianie nie-
których ustaw w związku z przeciwdziałaniem sytuacjom kryzysowym zwi-
ązanym z wystąpieniem COVID-19
Author: RCL
Creator: Microsoft® Word 2010
Producer: Microsoft® Word 2010; modified using iText 2.1.7 by
1T3XT
CreationDate: Sat Nov 28 11:39:52 2020 CST
ModDate: Sat Nov 28 11:40:01 2020 CST
Custom Metadata: no
Metadata Stream: yes
Tagged: yes
UserProperties: no
Suspects: no
Form: AcroForm
JavaScript: no
Pages: 18
Encrypted: no
Page size: 595.32 x 841.92 pts (A4)
Page rot: 0
File size: 407654 bytes
Optimized: no
PDF version: 1.5
```

```
(kali@vbox)-[~/Downloads/File]
$ pdftinfo D19910350Lj.pdf
Title: Akt prawny
Author: Władysław Baksza
Creator: Microsoft® Word 2013
Producer: Microsoft® Word 2013
CreationDate: Tue Oct 12 06:08:08 2021 CDT
ModDate: Tue Oct 12 06:08:08 2021 CDT
Custom Metadata: no
Metadata Stream: no
Tagged: yes
UserProperties: no
Suspects: no
Form: none
JavaScript: no
Pages: 351
Encrypted: no
Page size: 595.32 x 841.92 pts (A4)
Page rot: 0
File size: 2601654 bytes
Optimized: no
PDF version: 1.5
```

Oba zrzuty ekranu zawierają wymagane przez ćwiczenie informacje.

```
(kali@vbox)-[~/Downloads]  
$ ghex File.zip
```

Zadanie nr 3

Który wiersz zawiera informacje o pliku(rozszerzeniu)? Wiersz nr 1 - zawiera sygnaturę archiwum .zip

Sygnatura reprezentowanej nazwy pliku - 50 4B 03 04 - oznacza archiwum .zip

50 4B 03 04			kmz	
50 4B 05 06 (empty archive)	PK ⁰⁰⁰⁰⁰⁰⁰⁰		maff	zip file format and formats based on it, such as EPUB, JAR, ODF, OOXML
50 4B 07 08 (spanned archive)	PK ⁰⁰⁰⁰⁰⁰⁰⁰	0	msix	
	PK ⁰⁰⁰⁰⁰⁰⁰⁰		odp	
			ods	
			odt	
			nk3	

Open

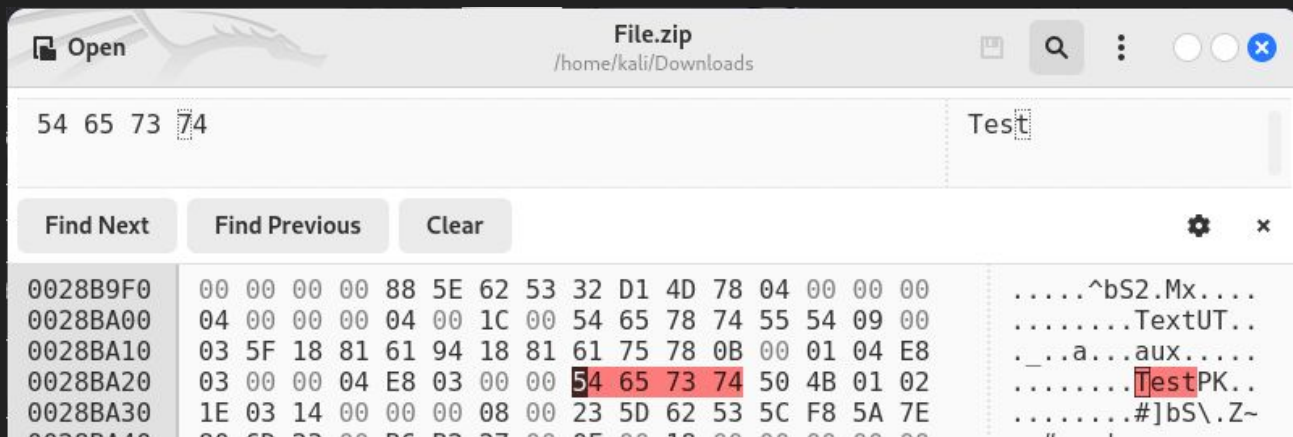
File.zip
/home/kali/Downloads

🔍 ⋮ ○ ○ ×

00000000	50 4B 03 04 14 00 00 00 08 00 23 5D 62 53 5C F8	PK.....#]bS\.
00000010	5A 7E 80 6D 23 00 B6 B2 27 00 0F 00 1C 00 44 31	Z~.m#...'.....D1

Czy w ramach pliku archiwum jesteśmy w stanie odczytać jego zawartość? Jeśli tak, to proszę ją wyświetlić, jeśli nie, to dlaczego?

Możemy odczytać nazwy plików, a czasem także ich treść (na zrzucie poniżej mamy treść pliku Text). Jest to możliwe wtedy, gdy dany plik jest zapisany metodą kompresji stored (czyli de facto bez kompresji).



Zadanie nr 4

```
(kali@vbox)-[~/Downloads]
$ dd if=/dev/zero of=sfile.raw count=100 bs=1M
100+0 records in
100+0 records out
104857600 bytes (105 MB, 100 MiB) copied, 0.049559 s, 2.1 GB/s

(kali@vbox)-[~/Downloads]
$ ghex sfile.raw
```

Jest to plik o wielkości 100MB, wypełniony zerami, a więc nie ma na początku żadnej sygnatury (w przeciwieństwie do poprzednio badanego pliku).

Następnie używamy polecenia `mkfs.fat` i widzimy efekt zmian jakie wywołało ono w pliku:



Na pliku sfile.raw został utworzony system plików FAT16

```
(kali@vbox)-[~/Downloads]
$ mkfs --type=ext4 sfile.raw
mke2fs 1.47.0 (5-Feb-2023)
sfile.raw contains a vfat file system
Proceed anyway? (y,N) y
Discarding device blocks: done
Creating filesystem with 102400 1k blocks and 25584 inodes
Filesystem UUID: 5f63d557-885b-49e5-bda5-fa5fc8f20947
Superblock backups stored on blocks:
    8193, 24577, 40961, 57345, 73729

Allocating group tables: done
Writing inode tables: done
Creating journal (4096 blocks): done
Writing superblocks and filesystem accounting information: done
```


Za pomocą dumpe2fs uzyskujemy:

- “magiczny numer” = `0xEF53`
- numer UUID = `3ea2d830-b557-4762-8099-c42ac407f982`
- wielkość bloku = `1024` Bajty
- liczbę wolnych bloków = `90319`
- typ sumy kontrolnej - `crc32c`
- ile wolnych bloków zawiera grupa nr 12 - `4095`

Ilość zużytych bloków w procentach: 11,8% `12081/102400`

Superbloki zostały utworzone na blokach: 8193, 24577, 40961, 57345, 73729

```
(kali@vbox)-[~/Downloads]
$ dumpe2fs sfile.raw
dumpe2fs 1.47.0 (5-Feb-2023)
```

```
(kali@vbox)-[~/Downloads]
$ fsck.ext4 -r -v -n sfile.raw
e2fsck 1.47.0 (5-Feb-2023)
sfile.raw: clean, 11/25584 files, 12081/102400 blocks
```

Zadanie nr 5

Plik lost+found służy do:

- przechowywania plików odzyskanych (np. za pomocą fsck po awarii, uszkodzeniu lub nieprawidłowym zamknięciu systemu plików),
- przechowywania plików niepoprawnie przypisanych do jakiegoś katalogu, lub plików, które straciły nazwę,

```
(kali@vbox)-[~/Downloads]
$ sudo losetup --find --show sfile.raw
[sudo] password for kali:
/dev/loop0

(kali@vbox)-[~/Downloads]
$ sudo mount /dev/loop0 /mnt/hgfs

(kali@vbox)-[/mnt/hgfs]
$ ls -li
total 12
11 drwx----- 2 root root 12288 Nov 22 14:39 lost+found
```

Wykonanie kolejnych instrukcji

Po “wyzerowaniu” zaledwie jednego bloku danych na naszym nośniku, nowoutworzony plik nie jest już widoczny w katalogu z danymi z obrazu.

```
(root@vbox)-[/mnt/hgfs]  
# sudo touch newitem
```

```
(root@vbox)-[/mnt/hgfs]  
# ls  
lost+found newitem
```

```
(root@vbox)-[/mnt/hgfs]  
# sudo dd if=/dev/zero of=/dev/loop0 count=1 bs=1024 seek=1  
1+0 records in  
1+0 records out  
1024 bytes (1.0 kB, 1.0 KiB) copied, 3.5078e-05 s, 29.2 MB/s
```

```
(root@vbox)-[/mnt/hgfs]  
# ls -li  
total 0
```

Próba odmontowania pliku - wystąpił błąd

Po kilku próbach i researchu dowiedziałem się, że był spowodowany tym, że próbowałem usunąć nośnik, w którym “byłem” używając terminala. Aby usunąć błąd wystarczyło przenieść się do jakiegoś innego katalogu.

```
(root@vbox)-[/mnt/hgfs]  
# sudo umount /mnt/hgfs  
umount: /mnt/hgfs: target is busy.
```

```
(kali@vbox)-[~]  
$ sudo umount /mnt/hgfs  
[sudo] password for kali:  
  
(kali@vbox)-[~]  
$ █
```

Próba zamontowania - błąd

Wygląda na to, że po odmontowaniu naszego "pendrive'a", przestał on być widoczny jako system plików. Powodem tego mogło być nadpisanie przeze mnie istotnych informacji zerami (jedna z poprzednich komend dd nadpisała ważne informacje, a skutki tego odczuliśmy dopiero przy próbie ponownego zamontowania /dev/loop0).

```
(kali@vbox)-[~] szym sposobem na  
$ sudo mount /dev/loop0 /mnt/hgfs  
mount: /mnt/hgfs: wrong fs type, bad option, bad superblock on /dev/loop0, missing codepage or helper program, or other error.  
dmesg(1) may have more information after failed mount system call.
```

Użyłem więc narzędzia do analizy i naprawy systemów plików - fsck.

Sukces - fsck znalazł błędy i je naprawił

```
(kali@vbox)-[~]
$ sudo fsck /dev/loop0
fsck from util-linux 2.40.2
e2fsck 1.47.1 (20-May-2024)
ext2fs_open2: Bad magic number in super-block
fsck.ext2: Superblock invalid, trying backup blocks...
Superblock needs_recovery flag is clear, but journal has data.
Recovery flag not set in backup superblock, so running journal anyway.
/dev/loop0: recovering journal
Pass 1: Checking inodes, blocks, and sizes
Pass 2: Checking directory structure
Pass 3: Checking directory connectivity
Pass 4: Checking reference counts
Pass 5: Checking group summary information
Block bitmap differences: +(8193--8450) +(24577--24834) +(40961--41218) +(57345--57602) +(73729--73986)
Fix<y>? yes
Free inodes count wrong for group #0 (1957, counted=1956).
Fix<y>? yes
Free inodes count wrong (25573, counted=25572).
Fix<y>? yes
Padding at end of inode bitmap is not set. Fix<y>? yes

/dev/loop0: ***** FILE SYSTEM WAS MODIFIED *****
/dev/loop0: 12/25584 files (0.0% non-contiguous), 12081/102400 blocks

(kali@vbox)-[~]
$ sudo mount /dev/loop0 /mnt/hgfs

(kali@vbox)-[~]
$
```


Wykonanie kolejnego polecenia z instrukcji

Akurat pokrywało się ono z moim sposobem na naprawę wcześniejszego błędu.

Poniżej przywrócona zawartość katalogu.

```
(kali@vbox)-[~]  
$ sudo fsck -f -y -b 8193 /dev/loop0 /mnt/hgfs  
fsck from util-linux 2.40.2  
e2fsck 1.47.1 (20-May-2024)  
e2fsck 1.47.1 (20-May-2024)  
fsck.ext2: Is a directory while trying to open /mnt/hgfs
```

```
The superblock could not be read or does not describe a valid ext2/ext3/ext4  
filesystem. If the device is valid and it really contains an ext2/ext3/ext4  
filesystem (and not swap or ufs or something else), then the superblock  
is corrupt, and you might try running e2fsck with an alternate superblock:
```

```
    e2fsck -b 8193 <device>  
or  
    e2fsck -b 32768 <device>
```

```
Pass 1: Checking inodes, blocks, and sizes  
Pass 2: Checking directory structure  
Pass 3: Checking directory connectivity  
Pass 4: Checking reference counts  
Pass 5: Checking group summary information  
Free inodes count wrong for group #0 (1957, counted=1956).  
Fix? yes
```

```
Free inodes count wrong (25573, counted=25572).  
Fix? yes
```

```
Padding at end of inode bitmap is not set. Fix? yes
```

```
Block bitmap differences: Group 1 block bitmap does not match checksum.  
FIXED.
```

```
/dev/loop0: ***** FILE SYSTEM WAS MODIFIED *****  
/dev/loop0: 12/25584 files (0.0% non-contiguous), 12081/102400 blocks
```

```
(kali@vbox)-[/mnt/hgfs]  
$ ls  
lost+found  newitem
```

Koniec

Dziękuję za uwagę